CHAPITRE 5 — LES MODÈLES & ELOQUENT ORM

Par Robert DIASSÉ

Comprendre, créer et utiliser les relations pour le projet fil rouge

1. Introduction : qu'est-ce qu'un Modèle en Laravel ?

Dans Laravel, **un modèle** est une classe PHP qui représente **une table** dans la base de données.

- La table **users** → modèle `User`
- La table **services** → modèle `Service`
- La table **reservations** → modèle `Reservation`

Eloquent ORM (Object Relational Mapping) transforme les données de MySQL en **objets PHP** faciles à manipuler.

Exemple concret :

Une entrée dans la table users devient :

```
$user = User::find(1);
echo $user->name;      // affiche le nom de l'utilisateur
```

Eloquent permet :

- d'insérer des données,
- de récupérer des données,
- de mettre à jour,
- de supprimer,
- de gérer des relations (1-N, N-N, 1-1).

2. Où sont stockés les modèles ?

Les modèles sont dans :

```
app/Models/
```

Laravel crée automatiquement `User.php`.
Il faudra créer `Service.php` et `Reservation.php`.

---

3. Créer les modèles du projet

Créer le modèle Service :

```
php artisan make:model Service
```

Créer le modèle Reservation :

```
php artisan make:model Reservation
```

Cela crée deux fichiers :

```
app/Models/Service.php
app/Models/Reservation.php
```

---

4. Le modèle User (déjà existant)

Ouvrir :

```
app/Models/User.php
```

Nous devons lui ajouter **une relation** avec les réservations.

Ajouter :

```php
public function reservations()
{
    return $this->hasMany(Reservation::class);
}
```

Explication :

Un user (patient) peut faire **plusieurs réservations**.

---

5. Le modèle Service

Ouvrir :

```
app/Models/Service.php
```

Ajouter :

```php
protected $fillable = [
    'titre',
    'description',
    'prix',
    'duree',
    'statut',
    'medecin_id'
];
```

Explication :

`$fillable` permet de spécifier quelles colonnes peuvent être remplies automatiquement via :

```
Service::create([...]);
```

Ajouter les relations :

```php
public function reservations()
{
    return $this->hasMany(Reservation::class);
}

public function medecin()
{
    return $this->belongsTo(User::class, 'medecin_id');
}
```

Logique :

- Un service peut être réservé plusieurs fois → **hasMany**
- Un service peut être attribué à un médecin → **belongsTo**

---

6. Le modèle Reservation

Ouvrir :

```
app/Models/Reservation.php
```

Ajouter :

```php
protected $fillable = [
    'user_id',
    'service_id',
    'date_reservation',
    'heure_reservation',
    'statut',
    'commentaire'
];
```

Relations :

```php
public function user()
{
    return $this->belongsTo(User::class);
}

public function service()
{
    return $this->belongsTo(Service::class);
}
```

Logique :

- Une réservation appartient à un utilisateur
- Une réservation appartient à un service

---

7. Résumé des relations (très important à mémoriser)

✔ User ↔ Reservation

- Un user → plusieurs réservations
- Une reservation → un seul user

✔ Service ↔ Reservation

- Un service → plusieurs réservations
- Une reservation → un seul service

✔ User ↔ Service (médecin)

- Un médecin → plusieurs services
- Un service → zéro ou un médecin

Graphique simplifié :

```
User (patient) 1 ---- n Reservation
Service        1 ---- n Reservation

User (médecin) 1 ---- n Service
```

---

8. Utilisation d'Eloquent

8.1 Récupérer toutes les réservations d'un patient

```
$user = auth()->user();
$reservations = $user->reservations;
```

8.2 Récupérer les réservations d'un service

```
$service = Service::find(1);
$reservations = $service->reservations;
```

8.3 Récupérer le médecin d'un service

```
$service = Service::find(1);
echo $service->medecin->name;
```

8.4 Créer une réservation

```
Reservation::create([
    'user_id' => auth()->id(),
    'service_id' => $service_id,
```

```
    'date_reservation' => $request->date_reservation,
    'heure_reservation' => $request->heure_reservation,
    'statut' => 'en_attente'
]);
```

8.5 Récupérer les réservations d'un médecin

```
$medecin = auth()->user();

$reservations = Reservation::whereHas('service', function ($query) use ($medecin)
{
    $query->where('medecin_id', $medecin->id);
})->get();
```

Très utile pour son dashboard médecin.

---

9. Les attributs protégés : `$fillable`

Eloquent empêche par sécurité de remplir toute la table d'un coup.

Avec :

```
Service::create($request->all());
```

Laravel refuse… sauf si les colonnes sont autorisées dans `$fillable`.

D'où leur importance dans chaque modèle.

---

10. Validation des relations

Test 1 — Récupérer les réservations d'un user

Dans Tinker :

```
php artisan tinker
```

```
$user = User::find(1);
$user->reservations;
```

Si cela retourne une collection → OK.

Test 2 — Récupérer les services d'un médecin

```
$med = User::where('role', 'medecin')->first();
$med->services;
```

Résultat attendu : liste de services.

Test 3 — Tester les relations inverses

```
$res = Reservation::first();
$res->user;
$res->service;
```

Résultat : objets User et Service.

---

11. Erreurs fréquentes et solutions

❌ Erreur : "Call to undefined relationship…"

Cause :

- méthode mal écrite
- nom de modèle incorrect
- mauvaise orthographe

Solution :

- vérifier le nom exact de la fonction relationnelle

❌ Erreur : "SQLSTATE[HY000]: No such column"

Cause :

- une migration n'a pas été exécutée
- une colonne est mal nommée

Solution :

```
php artisan migrate:fresh
```

❌ Erreur : “MassAssignmentException”

Cause :

- `$fillable` incomplet

Solution :

- ajouter la colonne dans `$fillable`

❌ Erreur : “Trying to get property of non-object”

Cause :

- objet null (relation inexistante)

Solution :

- vérifier avec `->exists()`
- ajouter `nullable()` dans la migration si nécessaire